

Utrecht University Repository

Title	Approximation Algorithms for Treewidth, Pathwidth, and Treedepth: A Short Survey
Authors	Bodlaender, Hans L.
Published in	Graph-Theoretic Concepts in Computer Science - 50th International Workshop, WG 2024, Revised Selected Papers;Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics);50th International Workshop on Graph-Theoretic Concepts in Computer Science, WG 2024
Publication Date	2025
Link	https://dspace.library.uu.nl/handle/1874/482731
Citation	Bodlaender, H L 2025, Approximation Algorithms for Treewidth, Pathwidth, and Treedepth : A Short Survey. in D Král' & M Milanič (eds), Graph-Theoretic Concepts in Computer Science - 50th International Workshop, WG 2024, Revised Selected Papers. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 14760 LNCS, Springer Science and Business Media Deutschland GmbH, pp. 3-18, 50th International Workshop on Graph-Theoretic Concepts in Computer Science, WG 2024, Gozd Martuljek, Slovenia, 19/06/24. https://doi.org/10.1007/978-3-031-75409-8_1 ;conference
Versions / License	Publisher version
Rights	https://www.uu.nl/en/university-library/license-and-reuse-conditions



Approximation Algorithms for Treewidth, Pathwidth, and Treedepth—A Short Survey

Hans L. Bodlaender^(✉) 

Department of Information and Computing Sciences, Utrecht University,
Utrecht, The Netherlands
h.l.bodlaender@uu.nl

Abstract. This short survey discusses old and new approximation algorithms for treewidth, and for the related parameters pathwidth and treedepth.

Keywords: Treewidth · Pathwidth · Treedepth · Approximation · Graph Algorithms

1 Introduction

Treewidth is one of the most studied graph parameters. The interest in this parameter comes from several applications, some in a theoretical setting, and some in an applied setting. Related graph parameters are pathwidth and treedepth. Each of these three parameters is NP-complete [3, 27, 41]. In this short survey, we discuss approximation algorithms for treewidth, pathwidth, and treedepth. We focus on algorithms with a guaranteed approximation ratio. We also discuss lower bound proofs for approximating the treewidth of graphs. For surveys on heuristic algorithms (without guaranteed ratios) for treewidth, see e.g., [14, 46].

This paper is organised as follows. In Sect. 2, we briefly introduce some notations. Section 3 introduces the parameters treewidth, pathwidth, and treedepth, and discusses equivalent definitions, and a relation with Gaussian elimination. Section 4 discusses algorithms that approximate the treewidth and give tree decompositions of the input graphs with a guaranteed bound on the width. Most of these algorithms have a similar shape, and are described in the setting of a general framework. The most recent algorithms use a different method. After that, Sect. 5 looks at lower bound results on polynomial time approximation of treewidth. Section 6 discusses approximations of treedepth and pathwidth. Some concluding remarks are made in Sect. 7.

This survey is on the occasion of the Test-of-Time award for the paper *Approximating Treewidth, Pathwidth, and Minimum Elimination Tree Height*. This paper was first presented at the 17th International Workshop on Graph Theoretic Concepts in Computer Science, WG 1991 [13] in Fischbachau, Germany, June 17–19, 1991; the award was given at the 50th International Workshop on Graph Theoretic Concepts in Computer Science, WG 2024 in Gozd Martuljek, Slovenia, June 19–21, 2024.

2 Some Notation

We sometimes use interval notation to denote intervals of integers, e.g., $[1, 3] = \{1, 2, 3\}$. When not differently specified, the variable n denotes the number of vertices of graph $G = (V, E)$.

For a graph $G = (V, E)$ and a vertex set $W \subseteq V$, the subgraph of G induced by W is $G[W] = (W, \{\{v, w\} \in E \mid v, w \in W\})$. With $G \setminus W$, we denote the graph $G[V \setminus W]$, i.e., the subgraph induced by all vertices not in W .

The operation to *eliminate* a vertex v in a graph G is the following: first, turn the neighbourhood of v into a clique (add an edge between each pair of non-adjacent neighbours of v), and then remove v . An example is given in Fig. 1.

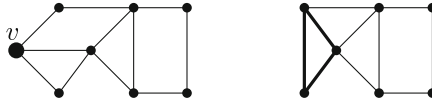


Fig. 1. A graph before and after eliminating vertex v

Most of approximation algorithms that we discuss in this paper can be divided in two categories: polynomial time approximation algorithms (with a running time of the form $O(n^{O(1)})$), and FPT approximation algorithms (with a running time of the form $O(f(k)n^{O(1)})$ where f is a computable function, k the parameter at hand). We say that an algorithm for parameter p has approximation ratio $f(p, n)$, or, equivalently, is a $f(p, n)$ -approximation, if there is a constant c , such that for each graph G with n vertices, the algorithm outputs a solution with value at most $f(p(G), n) \cdot p(G) + c$. E.g., if we output tree decompositions of width at most $4\text{tw}(G) + 3$, then we say that the algorithm has a approximation ratio 4; if we output tree decompositions of width at most $\sqrt{\log \text{tw}(G)} \cdot \text{tw}(G)$, then the algorithm has a approximation ratio $\sqrt{\log \text{tw}}$.

3 Graph Parameters

Treewidth. Several groups of authors independently invented graph parameters that are equivalent or similar to treewidth. Already in 1972, Bertelé and Briochi [7] introduced the equivalent concept of dimension. For some time, different terminologies were used, e.g., the notion of partial k -trees [3], until the concepts of treewidth and tree decompositions, introduced by Robertson and Seymour [44] became the standard terminology.

Definition 1. A tree decomposition of a graph $G = (V, E)$ is a pair $(\mathcal{T}, \mathcal{X})$, where $\mathcal{T} = (I, F)$ is a tree, and $\mathcal{X}$ is a collection of subsets of V , with for each $i \in I$ a set X_i , such that the following three properties hold:

1. $\bigcup_{i \in I} X_i = V$.

2. For all $\{v, w\} \in E$, there is an $i \in I$ with $v, w \in X_i$.
3. For all $v \in V$, the nodes $i \in I$ with $v \in X_i$ form a connected subtree of $\mathcal{T}$.

The width of a tree decomposition $(\mathcal{T}, \mathcal{X})$ is $\max_{i \in I} |X_i| - 1$. The treewidth of a graph G , denoted $\text{tw}(G)$, is the minimum width of a tree decomposition of G .

There are many definitions that are equivalent to treewidth; see [9] for an overview of several of these. Some are related to chordal graphs and the different equivalent characterisations of a chordal graph. An elimination scheme of a graph $G = (V, E)$ is a permutation π of the vertices. Suppose we eliminate the vertices of G one by one, in the order of elimination scheme π . The *width* of π is the maximum over all vertices of the degree when it is eliminated.

Theorem 1. *For a graph $G = (V, E)$ and integer k , the following are equivalent.*

1. $\text{tw}(G) \leq k$.
2. G is a subgraph of a chordal graph with maximum clique size at most $k + 1$.
3. G has an elimination scheme of width at most k .

Proof. (Sketch.) The equivalence of (1) and (2) follows from the fact that a chordal graph has a tree decomposition where each bag is a clique. The equivalence of (2) and (3) follows from the fact that chordal graphs have an elimination scheme where the neighbours of a vertex that is eliminated always form a clique. (For more background on chordal graphs, see [21].) $\square$

Definition 2. Let $G = (V, E)$ be a graph, $W \subseteq V$ a set of vertices, and $\alpha \in (0, 1)$ a real number between 0 and 1. A set $S \subseteq V$ is an α -balanced separator for W in G , if each connected component of $G \setminus S$ has at most $\alpha \cdot |W|$ vertices in W .

Central to many of the approximation algorithms discussed in this paper is the following well known result. The proof below is based on Robertson and Seymour [44, Theorem 2.5].

Theorem 2. *Let $G = (V, E)$ be a graph, and $W \subseteq V$ a set of vertices. There is a $\frac{1}{2}$ -balanced separator of W in G of size at most $\text{tw}(G) + 1$.*

Proof. (Sketch.) Take a tree decomposition of minimum width, $(\mathcal{T}, \mathcal{X})$. Take some node i_0 in $\mathcal{T}$. If X_{i_0} is not a $\frac{1}{2}$ -separator of W , then suppose W_1 is a connected component of $G \setminus X_{i_0}$ with $|W_1| > |W|/2$. Look at the subtrees of $\mathcal{T}$ obtained by removing i_0 from $\mathcal{T}$. All vertices in W_1 are in bags of exactly one of these subtrees – this follows from the properties of tree decompositions. Let i_1 be the neighbour of i_0 in this subtree. When i_1 is not a $\frac{1}{2}$ -separator of W , we repeat this step, and continue doing so until we find the desired balanced separator. After moving from i_j to i_{j+1} , we cannot go back to i_j in the next step, as this would give two disjoint subsets of W that both have size more than $|W|/2$. So the process stops, and we find a bag X_i that is a $\frac{1}{2}$ -separator of W . $\square$

Pathwidth. A *path decomposition* is a tree decomposition $(\mathcal{T}, \mathcal{X})$ with $\mathcal{T}$ a path. The *pathwidth* of a graph G , denoted $\text{pw}(G)$, is the minimum width of a path decomposition of G .

Clearly, the treewidth of a graph is at most its pathwidth. Several equivalent definitions of pathwidth exist, see [9] for a partial survey.

Treedepth. There are several equivalent definitions of treedepth. We give below a commonly used one.

Definition 3. An *elimination forest* of a graph $G = (V, E)$ is a forest $F = (V, F)$ with each tree in the forest rooted, such that for each edge $\{v, w\} \in E$, v is an ancestor of w in F or w is an ancestor of v in F . The *treedepth* of a graph G is the minimum depth over all elimination forests of G .

One equivalent notion is that of the *vertex ranking number* (see e.g., [10]). A vertex k -ranking of a graph $G = (V, E)$ is a function $f : V \rightarrow [1, k]$, such that for each $v, w \in V$ with $f(v) = f(w)$, on each path from v to w in G , there is a vertex x with $f(x) < f(v)$. The vertex ranking number of G is the smallest k for which G has a vertex k -ranking. Another is that of *minimum elimination tree height*: for an elimination ordering π of G , eliminate the vertices in order of π , and build a rooted forest by G by setting the parent of a vertex the lowest numbered neighbour it has when it is eliminated. Several other equivalent definitions can be found in a survey by Nešetřil and Ossona de Mendez [40], see also e.g., [39].

Theorem 3. For each graph G , the treedepth, vertex ranking number of minimum elimination tree height are equal.

Gaussian Elimination and Graphs. We now look at the relation between sparse symmetric matrices, Gaussian elimination, and graphs.

Suppose we have an n by n matrix A . We say that A is *symmetric* if for all $i, j \in [1, n]$, $a_{ij} = a_{ji}$. We can associate an undirected graph G_A with n vertices to a symmetric matrix A . The vertex set of G_A is $[1, n]$, i.e., we have a vertex for each pair of a row and its corresponding column. The edges are formed by the non-zero's: for all $i, j \in [1, n]$, we have an edge $\{i, j\}$ in G_A , if and only if $i \neq j$ and $a_{ij} \neq 0$.

If a matrix A has only non-zero entries (or only zero's on the main diagonal), then G_A is a complete graph. If we have a sparse matrix (a matrix with many entries that are 0), then G_A can reveal interesting structural insights.

In one step of the Gaussian elimination process, we create a column with only one non-zero by subtracting one of the rows a number of times from other rows. Suppose $a_{11} \neq 0$. Now, in the Gaussian elimination process, we can create a new matrix A' with a first column with all $a'_{i1} = 0$ for $i \neq 1$: for all $i \neq 1$ with $a_{i1} \neq 0$, we subtract $\frac{a_{i1}}{a_{11}}$ times the first row from the i th row. I.e., $a'_{ij} = a_{ij} - \frac{a_{i1}}{a_{11}} \cdot a_{j1}$. After this, the Gaussian elimination process continues, but the row and corresponding column of the first step are ignored; e.g., when we sweep with the first row, we continue on the $n - 1$ by $n - 1$ submatrix without the first row and first column.

Consider the sparse matrix A at the left side of (1); suppose entries are only 0 when explicitly shown as such. If we perform a Gaussian elimination step with the first row, then we obtain a 0 at the first element of the fourth and sixth row. Only the four entries marked with (*) change their value; all others are unchanged.

$$A = \begin{pmatrix} a_{11} & 0 & 0 & a_{14} & 0 & a_{16} \\ 0 & a_{22} & 0 & a_{24} & a_{25} & 0 \\ 0 & 0 & a_{33} & a_{34} & 0 & a_{36} \\ a_{14} & a_{24} & a_{34} & a_{44} & a_{45} & 0 \\ 0 & a_{25} & 0 & a_{45} & a_{55} & a_{56} \\ a_{16} & 0 & a_{36} & 0 & a_{56} & a_{66} \end{pmatrix} \rightarrow \begin{pmatrix} a_{11} & 0 & 0 & a_{14} & 0 & a_{16} \\ 0 & a_{22} & 0 & a_{24} & a_{25} & 0 \\ 0 & 0 & a_{33} & a_{34} & 0 & a_{36} \\ \mathbf{0} & a_{24} & a_{34} & (*) & a_{45} & (*) \\ 0 & a_{25} & 0 & a_{45} & a_{55} & a_{56} \\ \mathbf{0} & 0 & a_{36} & (*) & a_{56} & (*) \end{pmatrix} \quad (1)$$

It is interesting to see what happens to G_A when we eliminate a row with Gaussian elimination. We can create new non-zero elements: if we sweep with row k , then that a_{ij} only can become a new non-zero element when $a_{1k} \neq 0$ and $a_{kj} \neq 0$. In graph terms, this means that we can edges between neighbours of vertices of k . A In many applications (when matrix entries are real values from some application), it is not likely that a non-zero element becomes 0. Continuing on the submatrix without row and column corresponds to removing vertex k from the graph. Thus, one step of the Gaussian elimination process precisely corresponds to the elimination of a vertex (as described in Sect. 2). Figure 2 gives G_A , and the graph corresponding to the last five columns and rows of the matrix at the right side in (1).

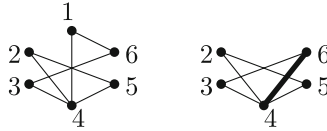


Fig. 2. Left: the graph G_A . Right: graph obtained from G_A by eliminating vertex 1.

The *minimum front-size* of matrix M is the minimum over all orderings in which we eliminate variables of the maximum number of non-zero's in one elimination step [12, 19]. For a fast execution of Gaussian elimination, it can be useful to eliminate rows in an order such that the front-size is small. Observe that this number of non-zero's equals the degree of the corresponding vertex when it is eliminated from $G[M]$, and thus the minimum front-size of M equals the treewidth of $G[M]$ —see e.g. [12] and Theorem 1.

The treedepth of a graph $G[M]$ reflects the time of a parallel algorithm for the Gaussian elimination: we can eliminate simultaneously a set of rows that corresponds to an independent set in $G[M]$ (and, in later steps, the graph resulting from $G[M]$ from earlier eliminations). Thus, the height of the elimination tree gives the number of such rounds of eliminating sets of rows (see Theorem 3 and [35, 36]).

4 Approximation of Treewidth

Many approximation algorithms for TREewidth have a similar structure. We first describe an algorithmic framework, that given an algorithm that finds balanced separators, gives an algorithm that approximates treewidth. After this, we briefly describe a number of approximation algorithms for treewidth by pointing out some specific characteristics and how they fit in the framework. At the end of this section, we discuss recent approximation algorithms for treewidth that are based on a different algorithmic paradigm.

An Algorithmic Framework. The main idea of the framework algorithm is that a tree decomposition is constructed top-down, where at each step, we use a balanced separator of the parent bag. Algorithm 1 gives pseudo-code of the framework. The framework has three steps that need further specification. In particular, different treewidth approximation algorithms implement Step 6 (where a balanced separator is found) in a different way. Possibly, the balanced separator algorithm can tell that the treewidth of the graph is larger than k , and we halt the entire procedure (Step 7). The bound in Step 3 can be set to the maximum width we aim to obtain (e.g., to $4k + 3$ if we aim to obtain a 4-approximation algorithm). We first will discuss the correctness of the procedure, and what approximation ratios we will obtain.

Algorithm 1. Treewidth-Approximate(G, W, k)

```

1: Input: Graph  $G = (V, E)$ , vertex set  $W \subseteq V$ .
2: Output: rooted tree decomposition with  $W$  subset of root bag, or ' $\text{tw}(G) > k$ '.
3: if  $|V| < \text{bound}$  then
4:   Output a tree decomposition with one bag  $V$ .
5: else
6:   Find a balanced separator  $S$  of  $W$  in  $G$ . ▷ How? See discussion!
7:   if No 'good' balanced separator exists then
8:     Output  $\text{tw}(G) > k$ ; stop.
9:   end if
10:  Compute the connected components of  $G \setminus S$ .
11:  Suppose the connected components of  $G \setminus S$  have vertex sets  $V_1, \dots, V_r$ .
12:  for  $i = 1, \dots, r$  do
13:     $W_i \leftarrow V_i \cap W$ .
14:    Recursively call Treewidth-Approximate( $G[V_i \cup S], W_i \cup S, k$ )
15:  end for
16:  Output a tree decomposition with new root bag  $i_r$  with  $X_{i_r} = W \cup S$ , and the
    disjoint union of all tree decompositions of the graphs  $G[V_i \cup S]$ , with  $i_r$  the parent
    of all root bags of these tree decompositions. ▷ See Figure 3.
17: end if

```

Lemma 1. *Suppose that at Step 6 of the Treewidth-Approximate algorithm, the algorithm either correctly determines that the treewidth of G is larger than k ,*

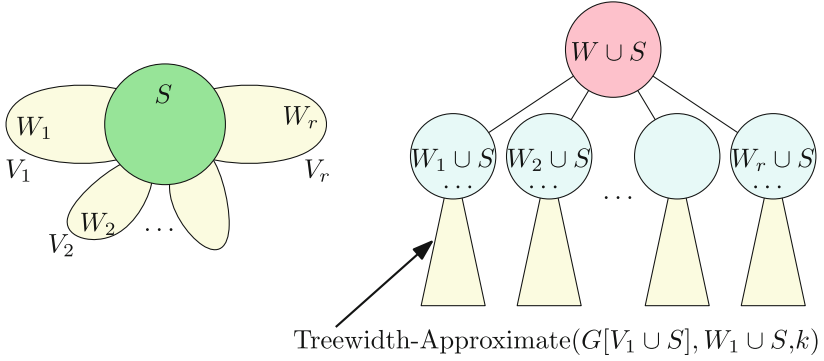


Fig. 3. Illustration of the last step of Treewidth-Approximate

or it finds an α -balanced separator of W in G of size $f(n, k)$. Then if we call $\text{Treewidth-Approximate}(G, W, k)$ with $|W| \leq \frac{1}{1-\alpha}f(n, k)$, then it either outputs a tree decomposition of G , with W a subset of the root bag of width at most $\max\{\text{bound}, (1 + \frac{1}{1-\alpha}) \cdot f(n, k)\}$ or correctly outputs that $\text{tw}(G) > k$.

Proof. It is not hard to verify that when the algorithm outputs a tree decomposition, this indeed satisfies the requirements of tree decompositions of G .

With induction, we have that for each recursive call done by the algorithm, the set in the second argument has size at most $\frac{1}{1-\alpha}f(n, k)$. This holds for the main call. If we recurse, then we have for each i that $|W_i| \leq \alpha \cdot |W|$, so $|W_i \cup S| \leq \alpha|W| + f(n, k) \leq \alpha \cdot \frac{1}{1-\alpha}f(n, k) + f(n, k) = \frac{1}{1-\alpha} \cdot f(n, k)$. Each bag is the union of a separator of size at most $f(n, k)$ and a set of size $\frac{1}{1-\alpha}f(n, k)$, so the width of the tree decomposition is at most $f(n, k) + \frac{1}{1-\alpha}f(n, k)$. $\square$

If the separator algorithm in Step 6 uses $T(n, k)$ time, then the entire procedure uses $O(n \cdot T(n, k))$ time, as the number of recursive calls in total is $O(n)$.

The First Treewidth Approximation. We now will discuss a number of different approximation algorithms for treewidth that fit (with variations) in the framework discussed above. The first such algorithm is based on the work by Robertson and Seymour [45] from the 1980s.

In the thirteenth paper in their monumental series of papers on Graph Minors, Robertson and Seymour [45] give an algorithm that given a graph G and an integer k , either decides that the branchwidth of G is at most k , or give a branch decomposition for G of width at most $3k$. The algorithm uses $O(3^{3k}n^2)$ time. For each graph G with branchwidth bw and treewidth tw , it holds that

$$\text{bw} - 1 \leq \text{tw} \leq \left\lfloor \frac{3}{2}\text{bw} \right\rfloor - 1.$$

The combination of these results directly implies a 4.5-approximation algorithm for treewidth that uses $O(3^{3\text{tw}}n^2)$ time. More close observation reveals that we have a 4-approximation algorithm.

Lemma 2. *Let $G = (V, E)$ be a graph with treewidth at most k , and $W \subseteq V$. Then we can partition V in three sets V_1, V_2, V_3 , with*

1. *no vertex in V_1 is adjacent to a vertex in V_3 ;*
2. *$|V_1 \cap W| \leq \frac{2}{3}|W|$; $|V_2 \cap W| \leq \frac{2}{3}|W|$;*
3. *$|V_2| \leq k + 1$.*

Proof. Let V_2 be a $\frac{1}{2}$ -balanced separator of W in G . We can group the connected components of $G \setminus V_2$ in two sets such that each contains at most two-thirds of the vertices in W . $\square$

The set V_2 functions as a $\frac{2}{3}$ -separator of W . Applying the framework, we will use sets W of size at most $\frac{1}{1-\frac{2}{3}} \cdot (k+1) = 3k+3$. Each vertex in W either is in V_1, V_2, V_3 : we try all 3^{3k+3} choices for this. For each, find a minimum size vertex separator S in the graph $G \setminus (W \cap V_2)$ that separates the vertices in $V_1 \cap W$ from the vertices in $V_2 \cap W$. If S has size larger than k , then $\text{tw}(G) > k$. S can be found by standard network flow techniques. As the size of the separator is at most k , we can employ at most k rounds of a flow augmentation in the Ford-Fulkerson algorithm, and thus each of the 3^{3k+3} tries can be done in $O(k \cdot (|V| + |E|)) = O(k^2 n)$ time.

Theorem 4 (Folklore, after Robertson, Seymour [45]). *There is an algorithm that, given a graph G , and integer k , either decides that $\text{tw}(G) > k$ or finds a tree decomposition of G of width at most $4k+3$ in $O(3^{3k} k^2 n)$ time.*

Various Separator Algorithms. We now discuss several other treewidth approximation algorithms, that all can be seen as special cases of Algorithm 1. In [12, 13], it was noted that we can use the following result.

Theorem 5 (Leighton, Rao, 1988 [33, 34]). *There is a polynomial time algorithm, that given a graph $G = (V, E)$ and a vertex set $W \subseteq V$, finds a $\frac{2}{3}$ -balanced separator of W in G of size $O(r \log n)$, where r is the minimum size of a $\frac{1}{2}$ -balanced separator of W .*

If we use the algorithm of Theorem 5 in the framework, then $r \leq \text{tw}(G) + 1$, and thus we find separators of size $O(\text{tw}(G) \cdot \log n)$, and the algorithm outputs tree decomposition of width $O(\text{tw}(G) \cdot \log n)$, i.e., we have a polynomial time algorithm with approximation ratio $O(\log n)$.

A number of papers give results that give FPT approximation or polynomial time approximation algorithms, with better approximation ratios or better running times.

- Matoušek and Thomas [37] use a probabilistic algorithm to find separators, resulting in an algorithm that uses $O(\text{tw}^{O(\text{tw})} n \log^2 n + n \log n \log p)$ time and has ratio 6, with a probability p to give an incorrect answer.
- Lagergren [31, 32] gives a parallel algorithm whose sequential variant uses $O(\text{tw}^{O(\text{tw})} \cdot n \log^2 n)$ time with a approximation ratio of 8.

- Reed [43] gives an algorithm that uses $O(\text{tw}^{O(\text{tw})} \cdot n \log n)$ time with a approximation ratio of 8.
- Amir [1, 2] gives several algorithms, including an FPT algorithm with ratio $3\frac{2}{3}$ and a polynomial time algorithm with ratio $O(\log \text{tw})$.
- Bouchitté et al. [16] use a separator approximation algorithm by Even [20] and obtain a polynomial time algorithm with approximation ratio $O(\log \text{tw})$.
- Feige, Hajiaghay, and Lee [23] give an improved algorithm for finding separators, which gives a polynomial time algorithm with approximation ratio $O(\sqrt{\log \text{tw}})$.
- Fomin et al. [25] have a fast $O(\text{tw}^7 n \log n)$ (polynomial) time algorithm with an approximation ratio $O(\text{tw})$.
- A version of Reed’s algorithm with a running time of the form $O(2^{O(\text{tw})} n \log n)$ was given by Belbasi and Fürer [5].

Double Balanced Separators. We now briefly discuss a variant that can help to speed up the algorithm, and has an interesting connection to treedepth. Suppose that we add to the separator found in Step 6 of Algorithm 1 a second balanced separator, but now one that is an α -balanced separator for the entire (current) vertex set V , for some $\alpha \in (0, 1)$. This increases the width of the obtained tree decomposition, but also ensures that the recursion depth of the algorithm, and hence the height of the tree $\mathcal{T}$ of the obtained tree decomposition are bounded by $O(\log n)$.

For instance, Reed [43] gives a clever algorithm that finds such a balanced separator of the vertex set of size $O(\text{tw})$ in $O(\text{tw}^{O(\text{tw})} n)$ time. When going in recursion, all but $O(\text{tw})$ vertices only participate in one branch, and with this, it follows that the total running time is bounded by $O(\text{tw}^{O(\text{tw})} n \log n)$.

We thus see that each graph G has a tree decomposition $(\mathcal{T}, \mathcal{X})$ of width $O(\text{tw}(G))$ with $\mathcal{T}$ a tree with height $O(\log n)$. With the following lemma it follows that $\text{td}(G) = O(\text{tw}(G) \cdot \log n)$.

Lemma 3. *Suppose G has a tree decomposition $(\mathcal{T}, \mathcal{X})$ of width k and with $\mathcal{T}$ a rooted tree of height t . Then the treedepth of G is at most $(k + 1) \cdot (t + 1)$.*

Proof. Take an arbitrary node of $\mathcal{T}$ as root. For each vertex v , let i_v be the highest node in $\mathcal{T}$ with $v \in X_{i_v}$. Assume some arbitrary ordering on the vertices V . We can rank the vertices top-down in the tree decomposition: set $f(v) = (k + 1) \cdot d + j$, if i_v has distance d to the root of $\mathcal{T}$, and v is the j th vertex in X_{i_v} . One can check that this is a vertex $(k + 1) \cdot (t + 1)$ -ranking of G , $\square$

An $O(c^{\text{tw}} n)$ 5-Approximation Algorithm. The first algorithm with a constant approximation ratio for treewidth with a running time of the form $c^{\text{tw}} n$ for some constant c was given in [11]. The algorithm uses a combination of techniques to speed up the steps in the framework. One of these is the following: with a variation of a technique from [8], we obtain (by running recursively the same algorithm on a smaller graph) a tree decomposition of the input graph G of width $10k + O(1)$. This tree decomposition can be used as a data structure that allows queries that allow to find balanced separators in less time. The paper also gives a 3-approximation algorithm with running time of the form $O(c^{\text{tw}} n \log n)$.

Local Improvement. In 2021, Korhonen [29] gave a novel algorithm that improved upon the result from [11] in several ways. His algorithm has running time of the form $O(c^{\text{tw}}n)$ and gives a 2-approximation for treewidth. The algorithm not only has a better approximation ratio, but also is much simpler and it has a significantly better constant c at the base of the exponent.

The algorithm also uses a new and very interesting technique. Instead of building a tree decomposition top-down as in Algorithm 1, Korhonen’s algorithm starts with a tree decomposition of larger width and then improves this tree decomposition in a number of successive steps.

First, by recursively calling the algorithm on a smaller graph using a modification of a technique from [8], we get a tree decomposition of width at most $4k + 3$. Then, as long as the current tree decomposition has a bag with $2k + 3$ or more vertices, the algorithm uses an improvement step: employing a generalisation of a result by Bellenbaum and Diestel [6], Korhonen shows that this large bag can be split and the vertices can be rearranged such that the resulting tree decomposition has fewer large bags. When the process stops, all bags have size at most $2k + 2$. Possibly, the algorithm finds that no improvement of a large bag is possible, and then it concludes that $\text{tw}(G) > k$.

Theorem 6 (Korhonen, 2021 [29]). *There is an algorithm using $2^{O(k)}n$ time, that given a graph G and integer k , either correctly outputs that $\text{tw}(G) > k$, or gives a tree decomposition of G of width at most $2k + 1$.*

A later paper by Korhonen and Lokshtanov [30] use this local improvement method to obtain for each $\epsilon > 0$, $(1 + \epsilon)$ -approximation algorithms that run in $\text{tw}^{\text{tw}/\epsilon} n^{O(1)}$ time.

5 Lower Bounds for Treewidth Approximation

In the previous section, we have seen a number of polynomial time and FPT approximation algorithms for the treewidth of graphs. Notably, we do not know of a polynomial time approximation algorithm for treewidth with a constant approximation ratio. Some complexity theoretic results indicate that the existence of such algorithms is unlikely.

The first published lower bound result was the relatively simple observation that polynomial time approximations with an additive term are unlikely. See [12] for similar results for pathwidth and treedepth.

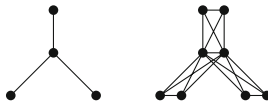


Fig. 4. Replace each vertex by a clique of size $c + 1$. In this example $c = 1$

Theorem 7 (Bodlaender, Gilbert, Hafsteinsson, Kloks [12,13]). *If $P \neq NP$, then for each $c \geq 0$, there is no polynomial time algorithm A_c , that given a graph G and integer k , either correctly outputs that the treewidth of G is more than k , or that the treewidth of G is at most $k + c$.*

Proof. Suppose on the contrary that we have such an algorithm A_c . Take an input $(G = (V, E), k)$ for the NP-complete TREEWIDTH problem. Build a graph H by replacing each vertex $v \in V$ by a clique with $c + 1$ vertices, with copies of v and w adjacent in H , if and only if v and w are adjacent in G . See Fig. 4. One can show that $\text{tw}(H) + 1 = (c + 1)(\text{tw}(G) + 1)$. So, if we apply A_c to H with target value $(k + 1)(c + 1) - 1$, then one of the following two cases holds. If A_c tells that the treewidth of H is at most $(k + 1)(c + 1) - 1 + c$, then the treewidth of H is at most $(k + 1)(c + 1) - 1$, and the treewidth of G is at most k . If A_c tells that the treewidth of H is larger than $(k + 1)(c + 1) - 1$, then the treewidth of G is larger than k . So, we have a polynomial time algorithm that decides on the treewidth of G , contradicting $P \neq NP$. $\square$

For many years, it was a major open problem whether approximations of treewidth with a constant approximation ratio existed. The first breakthrough was obtained by Wu et al. [47].

Theorem 8 (Wu, Austrin, Pitassi, Liu, 2014 [47]). *Suppose the Small Set Expansion hypothesis holds. Then, for any $\alpha > 1$, there is a $c > 0$, such that there is no polynomial time algorithm that for any given graph G with n vertices, either correctly outputs that $\text{pw}(G) \leq c \cdot n$, or correctly outputs that $\text{tw}(G) \geq \alpha \cdot c \cdot n$.*

The theorem implies that when we assume that the Small Set Expansion hypothesis holds, then there is no polynomial time approximation algorithm for TREEWIDTH and PATHWIDTH with a constant approximation ratio. Wu et al. [47] show that a similar inapproximability hold for a number of other graph parameters, but do not discuss treedepth.

The Small Set Expansion (SSE) hypothesis was introduced by Raghavendra and Steurer [42] in 2010. The hypothesis implies the Unique Games Conjecture, and is still topic of debate whether it should be considered to be holding. Very recently, Bonnet [15] gave an inapproximability result for treewidth for the much more common assumption that $P \neq NP$. He also gave a time bound based on the Exponential Time Hypothesis. The results are based on a clever transformation from 3-SATISFIABILITY.

Theorem 9 (Bonnet, 2024 [15]). *If $P \neq NP$, then there is no polynomial time approximation algorithm that approximates treewidth with approximation ratio 1.00005. If the Exponential Time Hypothesis holds, then there is a $\delta > 1$, such that an approximation algorithm with ratio δ takes time $2^{n^{1-o(1)}}$.*

6 Approximations of Treedepth and Pathwidth

All existing positive results for treewidth approximation directly transfer to approximations for treedepth and pathwidth with an additional factor $O(\log n)$,

as for each graph G , $\text{pw}(G) \leq \text{td}(G)$, and $\text{td}(G) = O(\log n \cdot \text{tw}(G))$. (For the former result, we can build a path decomposition by visiting the leaves of $\mathcal{T}$ ‘from left to right’, and for each, taking a bag with the leaf and all its ancestors in $\mathcal{T}$. For the latter result, see Lemma 3.)

Thus, in [13], $O(\log^2 n)$ -approximations of PATHWIDTH and TREEDPTH were given. Using the algorithm by Feige et al. [23], we get approximations for these problems with ratio $O(\sqrt{\log \text{tw}} \cdot \log n)$.

Some recent results give better approximations. Many of these are based on a ‘win-win strategy’: either we find a certain substructure in the graph that shows a lower bound, or we find an approximate solution. We start by a very simple example, that is based on a technique by Fellows and Langston [24, Theorem 9], and the discussion in [38, Section 6.2].

Lemma 4. *There is a linear time algorithm, that, when given a connected graph $G = (V, E)$ and an integer k , either correctly outputs that the treedepth of G is more than k , or gives an elimination forest for G of depth at most 2^k .*

Proof. Make a depth-first search spanning tree T of G . Such a tree has the property that each edge is between a pair with one vertex an ancestor of the other vertex. If T has depth at most 2^k , we output T ; otherwise, G has a path of length more than 2^k , so the treedepth of G is more than k ([38, Eq. 6.2].) $\square$

A related stronger result was given by Kawarabayashi and Rossman [28]. This result was further improved by Czerwiński, Nadara, and Marcin Pilipczuk [18], who showed that each graph of treedepth $\Omega(k^3)$ either contains as subgraph a path of length 2^k , or a subdivision of a full binary tree of depth k , or has treewidth at least k . Based on this structural insight, Czerwiński et al. [18] obtain a polynomial time approximation algorithm for treedepth. Note that the result also implies a ratio $\text{td} \log^{1.5} \text{td}$, as $\text{tw}(G) \leq \text{td}(G)$.

Theorem 10 (Czerwiński, Nadara, and Marcin Pilipczuk [18]). *There is a polynomial time algorithm that approximates the treedepth of an input graph G with a ratio $\text{tw} \log^{1.5} \text{tw}$.*

For pathwidth, an algorithm with bounds similar to those for treedepth given in Lemma 4 was obtained in 1996 by Cattell, Dinneen and Fellows [17]. This algorithm exploits that graphs of small pathwidth do not contain a large complete binary tree as subgraph.

Theorem 11 (Cattell, Dinneen, Fellows, 1996 [17]). *There is a polynomial time algorithm that when given a graph G and an integer k , either correctly outputs that the pathwidth of G is more than k , or gives a path decomposition of width at most 2^k .*

Recently, much stronger results were obtained by Groenland et al. [26], who give a combinatorial result that tells that each graph G has pathwidth at most $(\text{tw}(G) + 1)(h + 1)$ or has a subdivision of a complete binary tree of height $h + 1$ as

subgraph. The latter of these gives a lower bound h for the pathwidth. Groenland et al. [26] combine these insights with the treewidth approximation algorithm of Feige et al. [22] and obtain the following result on the approximation of pathwidth. Note that this result also implies a polynomial time approximation of pathwidth with a ratio $O(\text{pw}(G)\sqrt{\log(\text{pw}(G))})$.

Theorem 12 (Groenland, Joret, Nadara, Walczak, 2023 [26]). *There is a polynomial time algorithm that approximates the pathwidth of an input graph G with a ratio $O(\text{tw}(G)\sqrt{\log(\text{tw}(G))})$.*

Recently, Bansal et al. [4] gave a polynomial time algorithm to approximate the pathwidth of a graph with an approximation ratio of the form $O(\log^{1+o(1)} n)$. They use a different form of ‘recursive partitioning’ that is guided by a solution of a linear program.

7 Conclusions

In this short survey, older and newer developments on approximation algorithms for treewidth and the related parameters pathwidth and treedepth were discussed. In addition to theoretical studies with a focus to guarantees on running time and provable quality of approximations, several experimental studies on treewidth heuristics have been executed; for surveys, see [14] or [46]. Given the many theoretical and practical applications of the notions, these topics are of great interest. In this survey, we also did not discuss exact algorithms—of course, the running times of (in particular parameterized) approximation algorithms should be compared with that of exact algorithms. There also are several papers on treewidth approximation for special graph classes, but this short survey does not discuss these. We end with some open problems.

- The algorithms with currently best known (time or ratio) bounds for treewidth approximation are:
 - A polynomial time approximation algorithm with ratio $O(\sqrt{\log \text{tw}})$ by Feige et al. [23].
 - A 2-approximation algorithm with running time $2^{O(\text{tw})}n$ by Korhonen [29].
 - An $(1 + \epsilon)$ -approximation algorithm with running time $\text{tw}^{O(\text{tw}/\epsilon)}n^{O(1)}$ by Korhonen and Lokshtanov [30].
 - An $O(\text{tw}^7 n \log n)$ time algorithm with ratio $O(\text{tw})$ Fomin et al. [25].
 Are further improvements possible?
- Bonnet [15] showed a lower bound of 1.00005 for the approximation ratio for polynomial time treewidth approximation algorithms, conditional on $P \neq NP$. Can this bound be improved, and to what value? Is it possible to prove a similar result for pathwidth, and/or for treedepth?
- Are there polynomial time approximation algorithms for treedepth with a constant approximation ratio? Is it possible to show a negative result based on the Small Set Expansion hypothesis, similar to the results by Wu et al. [47]?

Acknowledgement. I thank John Gilbert, Hjalli Hafsteinsson, and Ton Kloks for great memories and great cooperation.

References

1. Amir, E.: Efficient approximation for triangulation of minimum treewidth. In: Breese, J.S., Koller, D. (eds.) *Proceedings of the 17th Annual Conference on Uncertainty in Artificial Intelligence, UAI 2001*, pp. 7–15. Morgan Kaufmann (2001)
2. Amir, E.: Approximation algorithms for treewidth. *Algorithmica* **56**, 448–479 (2010). <https://doi.org/10.1007/S00453-008-9180-4>
3. Arnborg, S., Corneil, D.G., Proskurowski, A.: Complexity of finding embeddings in a k -tree. *SIAM J. Algebraic Discrete Methods* **8**, 277–284 (1987). <https://doi.org/10.1137/0608024>
4. Bansal, N., Katzelnick, D., Schwartz, R.: Almost logarithmic approximation for cutwidth and pathwidth (2023). <https://doi.org/10.48550/ARXIV.2311.15639>
5. Belbasi, M., Fürer, M.: An improvement of Reed’s treewidth approximation. *J. Graph Algorithms Appl.* **26**(2), 257–282 (2022). <https://doi.org/10.7155/JGAA.00593>
6. Bellenbaum, P., Diestel, R.: Two short proofs concerning tree-decompositions. *Comb. Prob. Comput.* **11**(6), 541–547 (2002). <https://doi.org/10.1017/S0963548302005369>
7. Bertele, U., Brioschi, F.: *Nonserial Dynamic Programming*. Academic Press, New York (1972)
8. Bodlaender, H.L.: A linear time algorithm for finding tree-decompositions of small treewidth. *SIAM J. Comput.* **25**, 1305–1317 (1996). <https://doi.org/10.1137/S0097539793251219>
9. Bodlaender, H.L.: A partial k -arboretum of graphs with bounded treewidth. *Theoret. Comput. Sci.* **209**, 1–45 (1998). [https://doi.org/10.1016/S0304-3975\(97\)00228-4](https://doi.org/10.1016/S0304-3975(97)00228-4)
10. Bodlaender, H.L., et al.: Rankings of graphs. *SIAM J. Discrete Math.* **11**, 168–181 (1998). <https://doi.org/10.1137/S0895480195282550>
11. Bodlaender, H.L., Drange, P.G., Dregi, M.S., Fomin, F.V., Lokshtanov, D., Pilipczuk, M.: A $c^k n$ 5-approximation algorithm for treewidth. *SIAM J. Comput.* **45**(2), 317–378 (2016). <https://doi.org/10.1137/130947374>
12. Bodlaender, H.L., Gilbert, J.R., Hafsteinsson, H., Kloks, T.: Approximating treewidth, pathwidth, frontsize, and minimum elimination tree height. *J. Algorithms* **18**, 238–255 (1995). <https://doi.org/10.1006/JAGM.1995.1009>
13. Bodlaender, H.L., Gilbert, J.R., Hafsteinsson, H., Kloks, T.: Approximating treewidth, pathwidth, and minimum elimination tree height. In: Schmidt, G., Berghammer, R. (eds.) *WG 1991. LNCS*, vol. 570, pp. 1–12. Springer, Heidelberg (1992). https://doi.org/10.1007/3-540-55121-2_1
14. Bodlaender, H.L., Koster, A.M.C.A.: Treewidth computations I. Upper bounds. *Inform. Comput.* **208**, 259–275 (2010). <https://doi.org/10.1016/J.IC.2009.03.008>
15. Édouard Bonnet: Treewidth inapproximability and tight ETH lower bound (2024). <https://arxiv.org/abs/2406.11628>
16. Bouchitté, V., Kratsch, D., Müller, H., Todinca, I.: On treewidth approximations. *Discret. Appl. Math.* **136**, 183–196 (2004). [https://doi.org/10.1016/S0166-218X\(03\)00440-2](https://doi.org/10.1016/S0166-218X(03)00440-2)
17. Cattell, K., Dinneen, M.J., Fellows, M.R.: A simple linear-time algorithm for finding path-decompositions of small width. *Inform. Process. Lett.* **57**, 197–204 (1996). [https://doi.org/10.1016/0020-0190\(95\)00190-5](https://doi.org/10.1016/0020-0190(95)00190-5)
18. Czerwinski, W., Nadara, W., Pilipczuk, M.: Improved bounds for the excluded-minor approximation of treedepth. *SIAM J. Discret. Math.* **35**(2), 934–947 (2021). <https://doi.org/10.1137/19M128819X>

19. Duff, I.S., Reid, J.K.: The multifrontal solution of indefinite sparse symmetric linear equations. *ACM Trans. Math. Softw.* **9**, 302–325 (1983). <https://doi.org/10.1145/356044.356047>
20. Even, G., Naor, J., Rao, S., Schieber, B.: Fast approximate graph partitioning algorithms. *SIAM J. Comput.* **28**, 2187–2214 (1999). <https://doi.org/10.1137/S0097539796308217>
21. Even, S.: *Graph Algorithms*. Pitman, London (1979)
22. Feige, U., Hajiaghayi, M., Lee, J.R.: Improved approximation algorithms for minimum-weight vertex separators. In: *Proceedings of the 37th Annual Symposium on Theory of Computing, STOC 2005*, pp. 563–572. ACM Press (2005). <https://doi.org/10.1145/1060590.1060674>
23. Feige, U., Hajiaghayi, M., Lee, J.R.: Improved approximation algorithms for minimum weight vertex separators. *SIAM J. Comput.* **38**, 629–657 (2008). <https://doi.org/10.1137/05064299X>
24. Fellows, M.R., Langston, M.A.: On search, decision and the efficiency of polynomial-time algorithms. In: *Proceedings of the 21st Annual Symposium on Theory of Computing, STOC 1989*, pp. 501–512 (1989). <https://doi.org/10.1145/73007.73055>
25. Fomin, F.V., Lokshtanov, D., Saurabh, S., Pilipczuk, M., Wrochna, M.: Fully polynomial-time parameterized computations for graphs and matrices of low treewidth. *ACM Trans. Algorithms* **14**(3), 34:1–34:45 (2018). <https://doi.org/10.1145/3186898>
26. Groenland, C., Joret, G., Nadara, W., Walczak, B.: Approximating pathwidth for graphs of small treewidth. *ACM Trans. Algorithms* **19**(2), 16:1–16:19 (2023). <https://doi.org/10.1145/3576044>
27. Kashiwabara, T., Fujisawa, T.: An NP-complete problem on interval graphs. In: *Proceedings IEEE Symposium on Circuits and Systems*, pp. 82–83 (1979)
28. Kawarabayashi, K., Rossman, B.: A polynomial excluded-minor approximation of treedepth. *J. Eur. Math. Soc.* **24**(4), 1449–1470 (2022). <https://doi.org/10.4171/JEMS/1133>
29. Korhonen, T.: A single-exponential time 2-approximation algorithm for treewidth. In: *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021*, pp. 184–192. IEEE (2021). <https://doi.org/10.1109/FOCS52979.2021.00026>
30. Korhonen, T., Lokshtanov, D.: An improved parameterized algorithm for treewidth. In: Saha, B., Servedio, R.A. (eds.) *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023*, pp. 528–541. ACM (2023). <https://doi.org/10.1145/3564246.3585245>
31. Lagergren, J.: Efficient parallel algorithms for tree-decomposition and related problems. In: *Proceedings of the 31st Annual Symposium on Foundations of Computer Science, FOCS 1990*, pp. 173–182 (1990). <https://doi.org/10.1109/FSCS.1990.89536>
32. Lagergren, J.: Efficient parallel algorithms for graphs of bounded tree-width. *J. Algorithms* **20**, 20–44 (1996). <https://doi.org/10.1006/JAGM.1996.0002>
33. Leighton, F.T., Rao, S.: An approximate max-flow min-cut theorem for uniform multi-commodity flow problems with applications to approximate algorithms. In: *Proceedings of the 29th Annual Symposium on Foundations of Computer Science, FOCS 1988*, pp. 422–431. IEEE (1988). <https://doi.org/10.1109/SFCS.1988.21958>
34. Leighton, F.T., Rao, S.: Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. *J. ACM* **46**(6), 787–832 (1999). <https://doi.org/10.1145/331524.331526>

35. Liu, J.W.H.: The role of elimination trees in sparse factorization. *SIAM J. Matrix Anal. Appl.* **11**, 134–172 (1990). <https://doi.org/10.1137/0611010>
36. Liu, J.W.H.: The multifrontal method for sparse matrix solution: theory and practice. *SIAM Rev.* **34**(1), 82–109 (1992). <https://doi.org/10.1137/1034004>
37. Matoušek, J., Thomas, R.: Algorithms for finding tree-decompositions of graphs. *J. Algorithms* **12**, 1–22 (1991). [https://doi.org/10.1016/0196-6774\(91\)90020-Y](https://doi.org/10.1016/0196-6774(91)90020-Y)
38. Nešetřil, J., Ossona de Mendez, P.: *Sparsity*. AC, vol. 28. Springer, Heidelberg (2012). <https://doi.org/10.1007/978-3-642-27875-4>
39. Nešetřil, J., de Mendez, P.O.: Tree-depth, subgraph coloring and homomorphism bounds. *Eur. J. Comb.* **27**, 1022–1041 (2006). <https://doi.org/10.1016/J.EJC.2005.01.010>
40. Nešetřil, J., de Mendez, P.O.: On low tree-depth decompositions. *Graphs Comb.* **31**(6), 1941–1963 (2015). <https://doi.org/10.1007/S00373-015-1569-7>
41. Pothén, A.: The complexity of optimal elimination trees. Technical Report CS-88-13, Pennsylvania State University (1988). <https://www.cs.purdue.edu/homes/apothen/Papers/shortest-etree1988.pdf>
42. Raghavendra, P., Steurer, D.: Graph expansion and the unique games conjecture. In: Schulman, L.J. (ed.) *Proceedings of the 42nd ACM Symposium on Theory of Computing, STOC 2010*, pp. 755–764. ACM (2010). <https://doi.org/10.1145/1806689.1806792>
43. Reed, B.: Finding approximate separators and computing tree-width quickly. In: *Proceedings of the 24th Annual Symposium on Theory of Computing, STOC 1992*, pp. 221–228. ACM Press, New York (1992). <https://doi.org/10.1145/129712.129734>
44. Robertson, N., Seymour, P.D.: Graph minors. II. Algorithmic aspects of tree-width. *J. Algorithms* **7**, 309–322 (1986). [https://doi.org/10.1016/0196-6774\(86\)90023-4](https://doi.org/10.1016/0196-6774(86)90023-4)
45. Robertson, N., Seymour, P.D.: Graph minors. XIII. The disjoint paths problem. *J. Comb. Theor. Ser. B* **63**, 65–110 (1995). <https://doi.org/10.1006/JCTB.1995.1006>
46. Tamaki, H.: Experimental analysis of treewidth. In: Fomin, F.V., Kratsch, S., van Leeuwen, E.J. (eds.) *Treewidth, Kernels, and Algorithms*. LNCS, vol. 12160, pp. 214–221. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-42071-0_15
47. Wu, Y.L., Austrin, P., Pitassi, T., Liu, D.: Inapproximability of treewidth and related problems. *J. Artif. Intell. Res.* **49**, 569–600 (2014). <https://doi.org/10.1613/JAIR.4030>